

Chapter 2

REVIEW OF RELATED LITERATURE

TABLE OF CONTENTS

2.1 Introduction to the Review	4
2.2 Theoretical Framework	4
2.3 Historical Foundations of Natural-Language Interfaces	7
2.4 Natural-Language-to-Shell Translation	9
2.5 Graphical and Computer-Using Agents	11
2.6 Accessibility and Speech-Driven Interaction	15
2.7 Evaluation Environments and Benchmarks	16
2.8 Synthesis: Comparative Analysis and Research Gap	18
2.9 Review of the Methodological Literature	22
2.10 Summary	24
Notes	25
References	26

LIST OF TABLES

Table 2.1: Comparative analysis of surveyed systems along the 18
four-layer pipeline

LIST OF FIGURES

Figure 2.1: The four-layer interaction pipeline for language-mediated computing interfaces 5

2.1 Introduction to the Review

This chapter reviews the body of research relevant to the design and evaluation of a conversational session layer for the Linux desktop. The review spans nearly four decades of work on natural-language interfaces to computing systems, from the symbolic-artificial-intelligence consultants developed for the Unix shell in the late 1980s [1] to the large language model (LLM) agents that now operate shells, graphical applications, browsers, and mobile devices. Its purpose is threefold: to establish the conceptual vocabulary and technical foundations on which the present study builds, to characterize the current state of the field and its recurring limitations, and to locate precisely the gap that the proposed system, SynapseOS, is designed to fill.

2.2 Theoretical Framework

The heterogeneous systems surveyed in this chapter — shell translators, web and mobile agents, desktop automation frameworks, and accessibility interfaces — differ widely in platform, action space, and evaluation method, yet they address a common problem: converting a user's expressed intent into effects on a computing system. To compare them on common ground, this review adopts a *four-layer interaction pipeline* — Input, Reasoning, Action, and Operating-System Integration — as its theoretical framework.

The framework is adapted from the layered decomposition of large language model (LLM)-brained graphical user interface (GUI) agents proposed in the survey of Zhang and colleagues [18], generalized here from graphical agents specifically to language-mediated computing interfaces as a class. Figure 2.1 depicts the pipeline and situates the layers relative to one another.

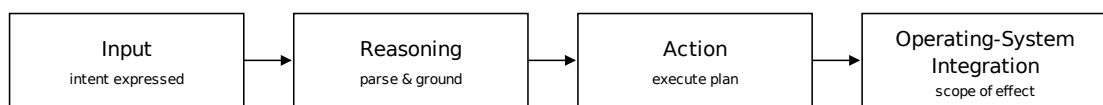


Figure 2.1. The four-layer interaction pipeline for language-mediated computing interfaces, adapted from the LLM-brained GUI-agent survey of Zhang and colleagues [18]. A user's intent enters at the Input layer, is parsed and grounded at the Reasoning layer, executed at the Action layer, and takes effect within a scope defined at the Operating-System Integration layer.

The *Input* layer concerns how a user expresses intent, whether by typed text, speech, or structured shortcuts. The *Reasoning* layer concerns how that intent is parsed into a plan and grounded into the affordances of the environment. The *Action* layer concerns how a grounded plan is executed — through command translation, through binding to native application programming interfaces (APIs), or through simulated keyboard and mouse operations. The *Operating-System Integration* layer concerns the scope of computing an approach can affect, from a single application to an entire operating system.

This framework is appropriate to the present study for three reasons. First, it is *modality-neutral*: because it abstracts over whether intent arrives as text, speech, or gesture and whether action is realized through shell commands, API calls, or simulated clicks, it accommodates the full span of surveyed systems without privileging any one interaction style — a necessary property for a review that spans command-line, graphical, and accessibility work. Second, it is *diagnostic*: locating each system's principal limitation at a specific layer — visual grounding as a Reasoning-and-Action failure, or narrow platform coverage as an Operating-System Integration limitation — turns an otherwise flat catalogue of systems into a structured comparison, and it is this layer-by-layer positioning that the comparative synthesis of §2.8 makes explicit. Third, it is *directly applicable* to the proposed system: the conceptual framework of SynapseOS presented in Chapter 3 instantiates this same pipeline — accepting natural-language intent at the Input layer, parsing it with a locally-hosted small language model at the Reasoning layer, translating it into shell commands executed through the standard toolchain at the Action layer, and spanning the full graphical session rather than a single application at the Operating-System Integration layer. The framework therefore serves both as the organizing lens for the literature and as the vocabulary in which the study's own contribution is later positioned.

Works were included in this review if they satisfy at least one of four criteria: they translate natural language into executable system actions; they operate graphical, web, or mobile interfaces through LLM-based reasoning; they address speech- or accessibility-driven interaction with computing systems; or they provide an

evaluation environment for such systems. Works concerned solely with model architecture or training, without a computing-interaction target, and general-purpose conversational systems without a system-control capability, were excluded as outside the scope of this study. The chapter first sets out the theoretical framework that organizes the review (§2.2), then proceeds thematically — historical foundations (§2.3), the shell and command-line thread most directly related to the proposed design (§2.4), graphical and computer-using agents (§2.5), accessibility and speech-driven interaction (§2.6), and evaluation environments (§2.7) — before synthesizing the surveyed works into a comparative analysis and an explicit statement of the research gap (§2.8). It closes with a review of the methodological literature that informs the study's evaluation design (§2.9) and a summary that motivates the methodology detailed in Chapter 3 (§2.10).

2.3 Historical Foundations of Natural-Language Interfaces

The aspiration to let users describe what they want in ordinary language and have a computer carry it out is not new. The earliest serious system in this tradition, the Berkeley Unix Consultant of Wilensky and colleagues [1], was already operational by the late 1980s. The Unix Consultant was an intelligent natural-language interface that allowed naive users to learn about the Unix operating system through conversation in ordinary English, addressing both the natural-language-frontend problem and the consultation problem, and thereby distinguishing itself from systems intended only as command translators. It was built on the KODIAK knowledge representation and incorporated a user-modeling component that

represented the user's knowledge state and a teaching component through which users could extend its knowledge base in natural language. The system demonstrated end-to-end natural-language consultation about Unix, successfully answering "how do I," definitional, and problem-solving questions. Critically, its authors identified knowledge acquisition as the principal bottleneck: extending the system to a new domain required substantial hand-engineering — a concern that directly foreshadows the contemporary problem of LLM-based agents requiring per-application grounding data.

The intervening decades saw natural-language interaction treated primarily as a supervised-learning problem. Representative of this pre-LLM era, the work of Li and colleagues on mapping natural-language instructions to mobile user-interface action sequences [19] trained dedicated phrase-extraction and grounding transformers on hand-annotated action sequences, achieving 70% partial accuracy on its benchmark but remaining bound to its purpose-built datasets and architectures. The decisive shift came with the introduction of in-context reasoning patterns for LLMs. ReAct, introduced by Yao and colleagues [20], collapsed the multi-stage supervised pipeline into a prompting strategy: the language model is prompted to alternate between free-text "Thought" steps that update its plan and "Action" steps that interact with an external environment, with observations fed back into subsequent reasoning. Implemented purely as a prompting strategy without fine-tuning, ReAct nonetheless outperformed imitation-learning and reinforcement-learning baselines that required thousands of training trajectories.

The historical significance of this transition is twofold, and it frames the remainder of this review. First, it explains why progress in the field has been so rapid since 2023: the methodological

foundations became portable across modalities and platforms in a way that the earlier dataset-bound approaches were not. Nearly every system surveyed in the sections that follow inherits from ReAct the convention of interleaving reasoning traces with environment-affecting actions. Second, this shared inheritance underscores a corresponding fragility: these systems inherit the failure modes of LLM reasoning — hallucinated actions, plan inconsistency, and brittleness on long-horizon tasks — alongside their capabilities, a concern that the evaluation literature reviewed in §2.7 quantifies.

2.4 Natural-Language-to-Shell Translation

The thread of the literature most directly related to the proposed system concerns the translation of natural language into shell commands. Because SynapseOS grounds user intent into commands executed through the standard Linux command-line toolchain, the accuracy, safety, and context-awareness of natural-language-to-Bash translation are of central relevance to its design.

Westenfelder and colleagues [2] address this translation problem directly. Their work constructs a manually verified test set of 600 instruction-command pairs and a training set of 40,939 pairs, and introduces a functional-equivalence heuristic that reaches 95% confidence in determining whether two Bash commands produce the same effect — allowing automated evaluation to move beyond exact-string matching to a measure of whether a generated command actually accomplishes the intended task. Their fine-tuned models substantially outperform zero-shot baselines, demonstrating that fine-tuning specifically on command-translation data is effective. The authors identify two persistent limitations: ambiguity in user

instructions, and the absence of system context — the contents of the current directory, the user's command history, and environment variables — at translation time. Integrating context-aware retrieval is identified as a direction, one that the persistent conversational session of SynapseOS is well positioned to supply.

Where translation accuracy is the concern of Westenfelder and colleagues, safety is the concern of Gyawali and colleagues, whose NaSh system [3] proposes guardrails for an LLM-powered natural-language shell for Linux. Published under the operating-systems classification, NaSh interposes a guardrail layer between the user's natural-language input and the LLM-generated command: the model proposes a candidate command, the guardrail layer evaluates it against safety, reversibility, and confirmation criteria, and the user receives a structured preview before execution. The authors note that LLM outputs may be unintended or unexplainable, requiring guardrails for user error recovery, and they leave as open work both the design of recovery and rollback mechanisms for irreversible system operations and the empirical user evaluation of the approach. These two open problems — reversibility-aware confirmation and empirical validation with users — are precisely the concerns that the confirmation gate, undo mechanism, and controlled user study of the present study are designed to address.

The feasibility of deploying such a system without reliance on a hosted frontier model is supported by the large-scale evaluation of Jacobs and colleagues [25], who assess a broad range of locally deployable open-source LLMs on Bash command generation. Their work provides an empirical anchor for the viability of running natural-language-to-Bash translation on local, open-weights models of modest size — the deployment posture adopted by SynapseOS for

reasons of privacy and offline operation. Taken together, the shell-and-command-line thread establishes that natural-language-to-Bash translation is tractable, that safety and reversibility are central rather than incidental concerns, and that local deployment is feasible; yet no surveyed shell-level system extends beyond the command line to the full graphical session, and none has been evaluated with users against the conventional workflow it would displace.

2.5 Graphical and Computer-Using Agents

A parallel and larger body of work addresses computer-using agents¹ — systems that operate graphical environments by issuing actions and observing results. Although SynapseOS operates at the command line rather than by simulating graphical interaction, this literature is essential context: it establishes the dominant architectural patterns, defines the recurring technical bottleneck of visual grounding², and, through its platform coverage, reveals the gap that motivates the present study. The work is organized here by the platform each system targets — web, mobile, and desktop — together with the vision-language foundation models on which many depend.

Web Agents

The web has been the most heavily studied environment for language-guided agents. Mind2Web, introduced by Deng and colleagues [7], is the first dataset for developing generalist web agents that follow language instructions across diverse real-world websites, comprising over two thousand open-ended tasks collected from 137 websites across 31 domains; its accompanying two-stage

model uses a small language model to filter candidate elements from large documents before invoking a larger model to choose among them. WebArena, introduced by Zhou and colleagues [8], provides a fully self-hosted, reproducible web environment of functional website clones and evaluates 812 long-horizon tasks for functional correctness rather than trajectory similarity, establishing a challenging benchmark on which contemporary models achieved completion rates far below human performance. Zheng and colleagues [9], in their SeeAct framework, decouple visual understanding from action: the model observes a screenshot, generates a textual plan, and a separate step grounds that plan onto page elements. Their central finding is diagnostic for the entire field — with oracle grounding provided, the model completes 51.1% of tasks on live websites, substantially outperforming a text-only baseline at 13.3%, but performance degrades sharply when grounding must itself be inferred, confirming visual grounding as the principal bottleneck.

Mobile Agents

Mobile-device control has produced large-scale datasets and progressively more autonomous agents. The pre-LLM work of Li and colleagues [19], discussed in §2.3, established the grounding problem for mobile interfaces. Android in the Wild, from Rawles and colleagues [10], contributes a dataset of 715,000 episodes spanning 30,000 unique instructions, in which actions are gestures at arbitrary screen coordinates rather than operations on indexed interface elements, exposing the challenge that actions must be inferred from visual appearance rather than from a structured model of the

interface. AppAgent, from Zhang and colleagues [11], operates Android applications through a simplified action space of taps and swipes and learns to use new applications through autonomous exploration or by observing human demonstrations, generating application-specific documentation that matched or exceeded human-written documentation. Mobile-Agent, from Wang and colleagues [12], adopts a purely vision-centric approach that requires no system metadata, using optical-character-recognition and icon-detection modules to locate interface elements — achieving broad applicability across mobile environments at the cost of vision-grounding errors that require recovery mechanisms.

Desktop and Cross-Application Agents

Desktop agents are the most direct graphical analogue to the present work, and their limitations are the most instructive. ScreenAgent, from Niu and colleagues [14], is a vision-language-model-driven agent for cross-application desktop control that operates in a plan-act-reflect cycle, specifying actions as coordinates and key sequences; its authors acknowledge that vision-only grounding is less reliable than approaches with API access. WinClick, from Hui and colleagues [15], targets Windows-desktop grounding and observes that most existing GUI agents rely on structured data formats such as the document object model or HyperText Markup Language, which are unavailable across general desktop environments; it therefore performs visual grounding directly from screenshots. UFO², the "Desktop AgentOS" of Zhang and colleagues [16], is the most ambitious desktop system surveyed: it fuses the native Windows accessibility framework with vision-based parsing in a

hybrid action mechanism, dispatching subtasks from a coordinating agent to application-specialized agents. Its authors argue that existing computer-using agents are hindered by shallow operating-system integration, fragile screenshot-based interaction, and disruptive execution, and they explicitly identify Linux and macOS coverage, sandboxing for safety, and full-operating-system conversational replacement — as opposed to automation layered atop the operating system — as the next directions. This assessment, from within the most advanced desktop-agent project surveyed, corroborates the gap the present study addresses.

Vision-Language Foundation Models

Underlying many of these agents are vision-language models trained specifically for interface understanding. CogAgent, from Hong and colleagues [13], is an eighteen-billion-parameter visual language model specialized for GUI understanding and navigation, using both low- and high-resolution image encoders to recognize the small icons and text typical of graphical interfaces. It supports the position that pure-vision GUI agents are viable when the underlying model is trained appropriately — but at a compute cost, and with the cross-platform generalization limitations common to the entire graphical-agent literature. Across web, mobile, and desktop, these systems demonstrate broad capability within a single platform class, while consistently identifying visual grounding as an unsolved bottleneck and trending toward hybrid execution that prefers structured access where available and falls back to vision.

2.6 Accessibility and Speech-Driven Interaction

A distinct thread of the literature approaches conversational computing from the perspective of accessibility, targeting users with motor, speech, or cognitive needs. This thread is relevant to the present study in two ways: it provides the most rigorous human-centered evaluations in the field, and it exhibits a limitation that is the mirror image of the mainstream agent literature, sharpening the opportunity the proposed system addresses.

VoicePilot, from Padmanabha and colleagues [5], is a framework for using LLMs as speech interfaces to physically assistive robots for individuals with motor impairments. Although its immediate target is robotic assistance rather than desktop computing, it is included here because it is the most rigorous human-centered evaluation in the LLM-speech-interface literature. The authors derive five concrete design guidelines from a study with motor-impaired participants, among them the importance of execution speed approaching that of a caregiver and the necessity of preview-and-confirm workflows — a guideline that directly informs the reversibility-based confirmation design of the present study. DexAssist, from Mehendale and Walishetti [6], targets web navigation by users with fine-motor impairments using two LLMs in tandem — one to interpret voice commands and a second to map the interpretation onto interface elements with fallback strategies — reducing the misinterpretation failures of single-model voice navigation on dense layouts. The architecture of van Dam [4] proposes retrofitting LLM-based conversational interfaces onto existing single-application graphical interfaces, exposing an application's navigation graph through a tool protocol and handling voice input through real-time

speech recognition; its author observes that most production applications were never designed with speech in mind, making such retrofitting an ongoing challenge.

More recent work continues to mature the components required for speech-driven interface control. Han and colleagues, in GUIRoboTron-Speech [22], advance automated GUI agents driven directly by speech instructions, while Park and colleagues, in R-VLM [23], contribute region-aware visual grounding for more precise identification of on-screen elements. What unites this thread is that it operates at the level of the individual application or interaction technique rather than at the level of the session itself. These approaches presuppose an underlying conversational system surface on which to run, yet no such surface exists at the session level. This complementarity is significant: a general-purpose conversational session layer would serve not only mainstream users but also provide the substrate that accessibility-specific configurations require — positioning accessibility not as the immediate target of the present study, which scopes it out for recruitment and ethics reasons, but as a natural and well-grounded extension of its contribution.

2.7 Evaluation Environments and Benchmarks

The rapid proliferation of computer-using agents has been accompanied by increasingly rigorous evaluation environments, and these establish both the current ceiling of the field's capability and the methods by which that capability is measured. The most consequential is OSWorld, from Xie and colleagues [17], a benchmark of 369 real-world computer tasks in a real-computer environment spanning Ubuntu, Windows, and macOS. Its headline result is stark

and frequently cited: humans achieve 72.36% success on these tasks, while the best evaluated model reaches only 12.24%. This wide gap, sustained across model families including GPT, Claude, Gemini, and Qwen, establishes that no current agent approaches human-level operating-system competence, with weaknesses concentrated in GUI grounding and operational knowledge. OSWorld is also directly relevant to the present study as the source of a real-task suite against which benchmark comparability can be established.

Complementing the benchmark literature, the survey of LLM-brained GUI agents by Zhang and colleagues [18] synthesizes the field through 2024, organizing it around the components of perception, planning, action, and memory, and providing the four-layer vocabulary adopted in §2.2 of this review. It enumerates the field's open challenges — accurate element localization, knowledge retrieval, long-horizon planning, and safety-aware execution control — and identifies among the gaps it leaves open the empirical, full-operating-system conversational replacement that the present study pursues. What the evaluation literature reveals, however, is a specific methodological limitation: rigorous as these benchmarks are, they measure the autonomous task completion of agents acting on a user's behalf, not the comparative performance of a human user working through a conversational interface versus a conventional graphical one. That distinction motivates the evaluation design reviewed in §2.9.

2.8 Synthesis: Comparative Analysis and Research Gap

Positioning the surveyed works against one another along the four-layer pipeline makes the structure of the field, and the gap within it, explicit. Table 2.1 summarizes the principal systems by coverage scope, action mechanism, evaluation approach, and key limitation.

Table 2.1. Comparative analysis of surveyed systems on conversational and LLM-mediated interfaces for personal computing, positioned by coverage scope, action mechanism, evaluation, and key limitation.

Ref.	System	Year	Coverage scope	Action mechanism	Evaluation	Key limitation
[1]	Unix Consultant	1988	Shell consultation	Natural-language Q&A (no execution)	Demonstration, case studies	Hand-engineered knowledge base
[2]	NL2Bash	2025	Shell (CLI)	Command translation	600-pair test set, fine-tuning	Limited system-context awareness
[3]	NaSh	2025	Shell (Linux)	Guarded command translation	Worked examples, design paper	No formal user evaluation
[4]	Multimodal GUI Arch.	2025	Single application	API binding + voice	Architectural prototype	Per-application retrofitting
[5]	VoicePilot	2024	Assistive robot	Voice + robot API	User study, motor-impaired participants	Robot domain, not the OS
[6]	DexAssist	2024	Web (e-commerce)	Dual-LLM voice + element mapping	Web-task evaluation	Narrow to e-commerce
[7]	Mind2Web	2023	Web (137 sites)	Element selection from filtered HTML	Dataset benchmark	Structured-data-only in original form
[8]	WebArena	2024	Web (self-hosted)	Functional task execution	Benchmark, low model baseline	Simulated, not live web
[9]	SeeAct	2024	Web (live sites)	Visual plan + grounding	51.1% with oracle grounding	Grounding hard without oracle
[10]	Android in the Wild	2023	Mobile (Android)	Coordinate gestures	Dataset, 715k episodes	Brittle to interface updates
[11]	AppAgent	2023	Mobile (10 apps)	Tap/swipe + element labels	50-task evaluation	Per-app knowledge acquisition

Ref.	System	Year	Coverage scope	Action mechanism	Evaluation	Key limitation
[12]	Mobile-Agent	2024	Mobile (vision-only)	OCR + icon detection + planning	Mobile-Eval benchmark	Vision-grounding errors
[13]	CogAgent	2024	Cross-platform GUI	Vision-language model (high-res)	SOTA on VQA / navigation	Compute cost of high-res inputs
[14]	ScreenAgent	2024	Cross-app desktop	Vision + mouse/keyboard	Plan-act-reflect evaluation	Vision-only grounding fragility
[15]	WinClick	2025	Desktop (Windows)	Visual grounding	WinSpot benchmark	Windows-specific
[16]	UFO ²	2025	Desktop (Windows)	Hybrid API + vision	System deployment, task benchmarks	Windows-only; automates atop the OS
[17]	OSWorld	2024	Ubuntu, Windows, macOS	Evaluation harness	369 tasks; 12.24% best vs 72.36% human	Reveals field-wide grounding gap
[19]	PixelHelp / seq2act	2020	Mobile (Pixel)	Two-stage transformer	ACL benchmark, three datasets	Pre-LLM; dataset-bound
[20]	ReAct	2023	Cross-domain	Thought-action-observation prompting	Multi-benchmark evaluation	Inherits LLM hallucination, brittleness
[22]	GUIRoboTron-Speech	2025	GUI (speech-driven)	Speech-instructed agent	Benchmark evaluation	Application, not session, level
[23]	R-VLM	2025	GUI grounding	Region-aware visual grounding	Grounding benchmark	Component, not full system
[25]	Local LLMs for NL2Bash	2026	Shell (CLI)	Open-source model evaluation	Large-scale model comparison	Evaluation, not a deployed system

Six patterns emerge from this comparison, each surfacing a recurring theme in the literature.

Bifurcation by Coverage Scope

The field is bifurcated by coverage scope. Shell-level work [2, 3] and single-application work [4, 5] sit at the narrow end; cross-application desktop, web, and mobile agents sit at the broad end but each within a single platform class. No surveyed system implements a working conversational replacement for the full graphical session that

has been empirically validated against the conventional desktop workflow it would displace. The Unix Consultant [1] came closest in ambition four decades ago, but its pre-LLM foundation prevented it from scaling beyond demonstration.

Execution Mechanism at the Action Layer

A second pattern concerns the execution mechanism at the Action layer. The literature divides among command translation, API binding, and GUI simulation, with a dominant trend toward hybrid execution [16, 4] that prefers structured access where available and falls back to vision. Vision-only systems [12, 14] achieve broad applicability at the cost of grounding fragility, while structured-access systems achieve precision at the cost of per-environment integration. Notably, the Unix Consultant operated purely at the consultation layer without executing actions — an approach that reflects an underappreciated design principle, that the agent that explains can be safer than the agent that acts, which the confirmation-gated design of the present study revisits.

Visual Grounding as the Principal Bottleneck

A third pattern is that visual grounding is the field's principal bottleneck, identified explicitly by both SeeAct [9] and the GUI-agents survey [18]. The 51.1% ceiling of GPT-4V with oracle grounding, its sharp degradation without, and the 12.24% best-model performance on OSWorld [17] are all attributable in significant part to grounding failure. This bottleneck is a property of the graphical-simulation approach specifically; a command-line approach that grounds intent

into shell commands rather than into on-screen pixel regions sidesteps it, at the cost of being bounded by what is expressible on the command line.

Scarcity of Comparative User Evaluation

A fourth pattern concerns evaluation method. Most surveyed systems are evaluated either through architectural demonstration [3, 4] or through curated task benchmarks [7, 8, 12, 17]. User studies are rare; only VoicePilot [5] conducts a substantive human-centered evaluation of LLM-based interaction, and no surveyed work compares its interface against the conventional graphical workflow for the same task on the same population. This is the empirical gap best filled by a controlled study of conversational interfaces.

The Operating-System Substrate

A fifth pattern concerns the operating-system substrate. Windows [15, 16], Android [10, 11, 12], and the web [7, 8, 9] are each heavily represented; Linux desktop coverage is conspicuously absent. NaSh [3] addresses Linux but only at the shell level, and OSWorld [17] includes Ubuntu but only as one of three evaluation platforms rather than as a primary deployment target. No surveyed system implements a working conversational session layer for the Linux desktop, despite Linux serving as the substrate for most developer and server-side workflows.

Shared Lineage and Its Fragility

A sixth pattern is the field's shared intellectual lineage and its attendant fragility. As established in §2.3, nearly every post-2023 system inherits the ReAct [20] convention of interleaving reasoning with action, which explains both the field's rapid progress and the pervasiveness of LLM-reasoning failure modes across it — a fragility that makes safety and recoverability mechanisms, rather than raw capability alone, decisive for any deployable system.

These patterns converge on a threefold research gap. *First*, no system in the literature replaces the conventional graphical session layer with a conversational one; existing systems automate atop the desktop shell rather than substituting for it — a limitation acknowledged from within the field's most advanced desktop project [16]. *Second*, Linux desktop coverage is absent despite Linux being the substrate for most developer and server-side workflows. *Third*, no existing work compares a conversational interface against the conventional graphical workflow on the same tasks with the same user population. SynapseOS — a conversational session layer for the Linux desktop that translates natural-language intent into shell commands under a reversibility-based confirmation gate, evaluated against each participant's own operating system — is positioned precisely against these three gaps.

2.9 Review of the Methodological Literature

Beyond the systems it proposes to build upon, the present study draws on a methodological literature that shapes how a conversational interface should be evaluated. The evaluation practices of the surveyed work fall into three categories, each with a characteristic limitation. Architectural demonstrations [3, 4] establish

feasibility but provide no performance evidence. Curated task benchmarks [7, 8, 17] provide reproducible, quantitative measures but assess the autonomous task completion of agents acting on a user's behalf rather than the performance of a human working through an interface. Human-centered user studies, exemplified by VoicePilot [5], provide the most direct evidence of an interface's value to real users but are rare in this literature and, where present, do not compare the proposed interface against the conventional workflow it would replace.

This critique motivates the methodological posture of the present study, which is grounded in established human-computer-interaction evaluation practice. The study adopts a *within-subjects* design, in which each participant completes tasks under both the SynapseOS condition and the condition of their own operating system, so that each participant serves as their own control and between-person variance is removed as a confound; the order of conditions is counterbalanced to distribute learning effects evenly. It further adopts an *expert-baseline* comparison — using each participant's most familiar environment as the comparison condition rather than a fixed alternative that may be unfamiliar — so that observed differences reflect interface quality rather than differential familiarity with the operating system, strengthening the ecological validity that the benchmark-only evaluations in the surveyed literature lack. To integrate the quantitative measures of task completion time and error rate with qualitative evidence of user experience, the study employs a *convergent mixed-methods* structure, collecting both concurrently and integrating them at the interpretation stage; the qualitative data, gathered in part through a think-aloud protocol, are analyzed using the thematic-analysis method of Braun and Clarke

[24], a widely adopted six-phase approach for identifying and reporting patterns in qualitative data. Design considerations surfaced by the reviewed systems inform the evaluation directly: the preview-and-confirm guideline of VoicePilot [5] and the reversibility-and-guardrail framing of NaSh [3] together motivate an explicit validation of the confirmation gate and undo mechanism as part of the study.

The specific instantiation of this methodology — the composition of the benchmark and custom task suite, the participant profile and screening criteria, the session procedure, the metrics and their computation, and the statistical and qualitative analysis plan — is presented in detail in Chapter 3.

2.10 Summary

This chapter reviewed the literature on natural-language and LLM-mediated interfaces to personal computing across nearly four decades, organized along a four-layer pipeline of Input, Reasoning, Action, and Operating-System Integration. The historical foundations established the long-standing aspiration to a natural-language interface to a real operating system [1] and the methodological shift, driven by in-context reasoning [20], that made the current generation of systems possible. The shell-and-command-line thread most directly related to the proposed design demonstrated that natural-language-to-Bash translation is tractable [2], that safety and reversibility are central concerns [3], and that local deployment is feasible [25], while leaving the extension from shell to session, and empirical user evaluation, as open work. The graphical-agent literature across web, mobile, and desktop [7–16] established the field's architectural patterns and its recurring bottleneck of visual grounding, while the

accessibility thread [4, 5, 6, 22, 23] revealed a complementary limitation that a general conversational session layer would resolve. The evaluation literature [17, 18] quantified the wide gap between current agents and human competence and exposed a methodological gap: interfaces are not evaluated comparatively against the conventional workflow with real users.

Synthesized, the surveyed works reveal a threefold gap: no conversational replacement for the graphical session layer, no coverage of the Linux desktop, and no controlled comparison of a conversational interface against the conventional workflow across user populations. The present study addresses all three through SynapseOS, a conversational session layer for the Linux desktop evaluated by controlled, within-subjects comparison against each participant's own operating system. The methodology through which this evaluation is conducted is the subject of Chapter 3.

Notes

1. A computer-using agent is an artificial-intelligence system that operates a computing environment by issuing actions and observing their results. A GUI agent is a computer-using agent whose action space consists of operations on a graphical user interface — typically clicks, key presses, scrolls, and text entry — usually driven by a large language model or a multimodal (vision-language) model that additionally accepts images as input.
2. Grounding is the problem of mapping a model's textual plan onto a concrete operation in the environment. Visual grounding is the specific subproblem of identifying the on-screen pixel region corresponding to a referenced interface element — repeatedly identified in the surveyed literature as the principal bottleneck for graphical agents, and one that a command-line approach avoids by grounding intent into shell commands rather than into screen coordinates.

References

- [1] Wilensky, R., Chin, D. N., Luria, M., Martin, J., Mayfield, J., & Wu, D. (1988). The Berkeley Unix Consultant Project. *Computational Linguistics*, 14(4), 35-84. <https://aclanthology.org/J88-4003/>
- [2] Westenfelder, F., Hemberg, E., Tulla, M., Moskal, S., O'Reilly, U.-M., & Chiricescu, S. (2025). LLM-Supported Natural Language to Bash Translation. *arXiv:2502.06858*. <https://arxiv.org/abs/2502.06858>
- [3] Gyawali, B. R., Achalla, S., Kallas, K., & Kumar, S. (2025). NaSh: Guardrails for an LLM-Powered Natural Language Shell. *arXiv:2506.13028*. <https://arxiv.org/abs/2506.13028>
- [4] van Dam, H. G. W. (2025). A Multimodal GUI Architecture for Interfacing with LLM-Based Conversational Assistants. *arXiv:2510.06223*. <https://arxiv.org/abs/2510.06223>
- [5] Padmanabha, A., Yuan, J., Gupta, J., Karachiwalla, Z., Majidi, C., Admoni, H., & Erickson, Z. (2024). VoicePilot: Harnessing LLMs as Speech Interfaces for Physically Assistive Robots. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology (UIST '24)*. ACM. <https://doi.org/10.1145/3654777.3676401>
- [6] Mehendale, S., & Walishetti, A. (2024). DexAssist: A Voice-Enabled Dual-LLM Framework for Accessible Web Navigation. *arXiv:2411.12214*. <https://arxiv.org/abs/2411.12214>
- [7] Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., & Su, Y. (2023). Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing*

Systems 36 (NeurIPS 2023).

<https://arxiv.org/abs/2306.06070>

- [8] Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2024). WebArena: A Realistic Web Environment for Building Autonomous Agents. In *Proceedings of ICLR 2024*. <https://arxiv.org/abs/2307.13854>
- [9] Zheng, B., Gou, B., Kil, J., Sun, H., & Su, Y. (2024). GPT-4V(ision) is a Generalist Web Agent, if Grounded. In *Proceedings of ICML 2024*. <https://arxiv.org/abs/2401.01614>
- [10] Rawles, C., Li, A., Rodriguez, D., Riva, O., & Lillicrap, T. (2023). Android in the Wild: A Large-Scale Dataset for Android Device Control. In *Advances in Neural Information Processing Systems* 36 (NeurIPS 2023). <https://arxiv.org/abs/2307.10088>
- [11] Zhang, C., Yang, Z., Liu, J., Han, Y., Chen, X., Huang, Z., Fu, B., & Yu, G. (2023). AppAgent: Multimodal Agents as Smartphone Users. *arXiv:2312.13771*. <https://arxiv.org/abs/2312.13771>
- [12] Wang, J., Xu, H., Ye, J., Yan, M., Shen, W., Zhang, J., Huang, F., & Sang, J. (2024). Mobile-Agent: Autonomous Multi-Modal Mobile Device Agent with Visual Perception. *arXiv:2401.16158*. <https://arxiv.org/abs/2401.16158>
- [13] Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Zhang, Y., Li, J., Xu, B., Dong, Y., Ding, M., & Tang, J. (2024). CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of CVPR 2024*. <https://arxiv.org/abs/2312.08914>

- [14] Niu, R., Li, J., Wang, S., Fu, Y., Hu, X., Leng, X., Kong, H., Chang, Y., & Wang, Q. (2024). ScreenAgent: A Vision Language Model-driven Computer Control Agent. *arXiv:2402.07945*.
<https://arxiv.org/abs/2402.07945>
- [15] Hui, T., Zhang, Y., Jiang, B., Sun, J., Cheng, F., Lin, X., & Yang, Y. (2025). WinClick: GUI Grounding with Multimodal Large Language Models. *arXiv:2503.04730*.
<https://arxiv.org/abs/2503.04730>
- [16] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). UFO²: The Desktop AgentOS. *arXiv:2504.14603*.
<https://arxiv.org/abs/2504.14603>
- [17] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., & Yu, T. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*.
<https://arxiv.org/abs/2404.07972>
- [18] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024). Large Language Model-Brained GUI Agents: A Survey. *arXiv:2411.18279*. <https://arxiv.org/abs/2411.18279>
- [19] Li, Y., He, J., Zhou, X., Zhang, Y., & Baldridge, J. (2020). Mapping Natural Language Instructions to Mobile UI Action Sequences. In *Proceedings of ACL 2020*, 8198–8210.
<https://doi.org/10.18653/v1/2020.acl-main.729>

- [20] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of ICLR 2023*.
<https://arxiv.org/abs/2210.03629>
- [22] Han, W., Zeng, Z., Huang, J., Jiang, S., et al. (2025). GUIRoboTron-Speech: Towards Automated GUI Agents Based on Speech Instructions. *arXiv:2506.11127*.
<https://arxiv.org/abs/2506.11127>
- [23] Park, J., Tang, P., Das, S., Appalaraju, S., Singh, K. Y., Manmatha, R., & Ghadar, S. (2025). R-VLM: Region-aware Vision Language Model for Precise GUI Grounding. In *Findings of the Association for Computational Linguistics: ACL 2025*, 9669–9685. <https://doi.org/10.18653/v1/2025.findings-acl.501>
- [24] Braun, V., & Clarke, V. (2006). Using thematic analysis in psychology. *Qualitative Research in Psychology*, 3(2), 77–101. <https://doi.org/10.1191/1478088706qp063oa>
- [25] Jacobs, J., Lapon, J., & Naessens, V. (2026). Local LLMs for NL2Bash: A Large-Scale Open-Source Model Evaluation for Bash Command Generation. *NDSS Workshop on LLM Assisted Security and Trust Exploration (LAST-X 2026)*.
<https://www.ndss-symposium.org/wp-content/uploads/lastx2026-49.pdf>